LF09:08:Exkurs: Bitweise Verknüpfungen bzw. bitweise und logische Operatoren

0) Exkurs: Bitweise Verknüpfung: Kontext [→ ZP:Sheet:2]

Anknüpfung: Was bedeutete & im Kontext einer **MAC-Adresse** und ihrem **OUI**?

1) Bitweise Operatoren [→ ZP:Sheet:3]

Syntax: `NOT :- '!', AND :- '&', OR :- '|'`

Abbildung: `{0,1}` → `{0,1}`

	NOT [!]	OR [|]	AND [&]
R1	`!1 → 0`	`1|1 → 1`	`1&1 → 1`
R2	`!0 → 1`	`1|0 → 1`	`1&0 → 0`
R3		`0|1 → 1`	`0&1 → 0`
R4		`0|0 → 0`	`0&0 → 0`

Unterschied von Zahl und Darstellung [→ ZP:Sheet:4]

- Integer bleibt Integer - unabhängig von der Darstellung!
- Deshalb: Vor bitweiser Verknüpfung keine gesonderte Umwandlung 'dezimal → binär' nötig.

Vorführbeispiele:

- bitweises NICHT: `!(0111)` → `1000`
- bitweises ODER: `|(0101,0010)` → `0111` (= lax gesagt: *bitweise verodern*)
- bitweises UND: `&(0110,1011)` → `0010` (= *ausmaskieren* bzw. lax gesagt: *bitweise verundern*)

vgl. [https://de.wikipedia.org/wiki/Bitweiser_Operator]https://de.wikipedia.org/wiki/Bitweiser_Operator()

---

3) Logische Operatoren in Computersprachen [→ ZP:Sheet:5]

Die Definitionen ähneln sich:

Syntax: `NOT :- '!', AND :- '&&', OR :- '||'` Abbildung: `{T,F}`→ `{T,F}`

 CC-BY-4.0

	NOT [!]	OR [||]	AND [&&]
R1	! T → F	T || T → T	T && T → T
R2	! F → T	T || F → T	T && F → F
R3		F || T → T	F && T → F
R4		F || F → **F**	F && F → F

5) Logische Operatoren in der Logik [→ ZP:Sheet:6]

- werden mit Wahrheitswertetabellen definiert.
- mit 'ausmultiplizierten' Kombinationen

```
| p | q | ! p | ! q | p || q | p && q |
|---|---|-----|-----|--------|--------|
| w | w |  f  |  f  |   w    |   w    |
| w | f |  f  |  w  |   w    |   f    |
| f | w |  w  |  f  |   w    |   f    |
| f | f |  w  |  w  |   f    |   f    |
```

Anmerkung:

- v steht für oder
- und = (1w+3f) ist die strukturelle Opposition von oder (3w+1f)
- darum ist das logische Symbol für und das umgekehrte v

Lesehinweis: die logischen Zeichen sind im Font dieses Textes nicht darstellbar, nur in der Zen-Präsentation, Sheet 6. Sorry.

---

6) Leseaufgaben für Programmierer:

ÜBUNG **LF09:08:Bit-Operatoren:01**

☐ Beschreiben Sie, was [→ ZP:Sheet:7] besagt.

Lösung:

A) [→ ZP:Sheet:7]

- soll dieselbe logische Verknüpfung zweier Aussagen sein, einmal in Python, einmal in C/C++

 CC-BY-4.0

- ist es tatsächlich aber nicht: C++-Beispiel testet, ob bei 'x / 2' Rest bleibt, wenn ja : 'durch 2 teilbar'. Ein Rest bleibt aber: wenn nicht durch 2 teilbar!

```python
# PYTHON-Example:
x=2
if x % 2 == 0 and x % 3 == 0 :
  print("durch 2 und 3 teilbar")
else:
  print("nicht durch 2 und 3 teilbar")
```

```cpp
/* C++ Example */
int x=2;
if ((x % 2) && (x % 3)) {
  cout << "durch 2 und 3 teilbar - hmmh?";
}
else {
  cout << "nicht durch 2 und 3 teilba - hmmh?";
}
```

ÜBUNG **LF09:08:Bit-Operatoren:02**

☐ Beschreiben Sie, was [→ ZP:Sheet:8] besagt.

Lösung:

(B) [→ ZP:Sheet:8] enthält unerfüllbare Bedingung**

```python
# PYTHON-Riddle:
x=2
if (((x % 2) AND ((4 & x) == 1)) :
  print("logical revolution!")
else:
  print("häh?")
```

- `2 % 2` ist niemals true, weil `%-Rest = 0`
- `4 & 2` == `0b0100 & 0b0010` == `0b0000` : ergo nicht 1 : ergo false
- if-statement braucht logischer Verknüpfung wegen nur ersten Test auszurechnen

---

ÜBUNG **LF09:08:Bit-Operatoren:03**

☐ Bitte berechnen Sie `168 & 128`.
☐ Bitte berechnen Sie `(2==2 && "Birne"=="Apfel")`

 CC-BY-4.0

Lösung:

- `168 & 128` → `(128|40)& 128` → `128`
- (2==2 → `T`) && (`"Birne"`==`"Apfel"` → `F`)) → `(T && F)` → `F`
- Entscheidung erst im letzten logischen Term

 CC-BY-4.0